

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

Guardián de la Lluvia

Informe Técnico de Arquitectura, Patrones de Diseño y Modelado de Software

Integrantes:

Rodrigo Escudero

Ariel Fuenzalida

Matías Santelices

Octubre de 2026

Índice

Lista de figuras **2**

Lista de tablas **3**

Introducción **4**

1 GM-1: ¿Qué experimenta el jugador? **5**

2 GM-2: ¿Cómo se transformó el código base? **9**

3 Diagramas de clases (GM-3) **12**
3.1 Dominio: partida, lluvia y controles 13
3.2 Sistemas: presentación, combate y perfil 14

4 GM-4: ¿Qué abstracciones definen el comportamiento? **15**

5 GM-5: ¿Cómo se protegen los datos y se organiza el código? **17**

6 GM-6: ¿Por qué centralizar la configuración y el audio? (Singleton) . **19**

7 GM-7: ¿Cómo se desacoplan los controles de entrada? (Strategy) **21**

8 GM-8: ¿Cómo se define el esqueleto de la caída y captura? (Template Method) **23**

9 GM-9: ¿Cómo se construyen niveles complejos de forma legible? (Builder) **25**

10 Control de Versiones y Trazabilidad en GitHub (GM-10) **28**

11 Recorrección de Guardián de la Lluvia **29**
11.1 Strategy y Template Method 29
11.2 Singleton y Builder 29
11.3 Luma y las patrullas de la nave 30

Índice de figuras

Figura 1.1 Capturas del juego mostrando la evolución de biomas y la batalla climática final 7
Figura 1.2 Modalidades adicionales: Tienda de desbloqueos y Versus local en pantalla dividida 8
Figura 6.1 Diagrama UML del patrón Singleton aplicado a la configuración global y audio 20
Figura 7.1 Diagrama UML del patrón Strategy aplicado al control de movimiento 22
Figura 8.1 Diagrama UML del patrón Template Method en la jerarquía Gota .. 24
Figura 9.1 Diagrama UML del patrón Builder con Director en Dificultad 27

Índice de tablas

Tabla 1.1 Tipos de elementos, sus efectos inmediatos y variación de estado 6
Tabla 2.1 Comparativa entre el prototipo base y la solución final implementada (GM) 10

Introducción

El presente informe técnico expone la arquitectura, modelado conceptual, patrones de diseño e implementación del videojuego **GameLluvia: Guardián de la Lluvia**, un software interactivo desarrollado en el lenguaje de programación Java utilizando el framework multimedia **LibGDX**.

A partir de un prototipo base simple —el cual presentaba limitaciones de acoplamiento, fallos en la eliminación de elementos e iteraciones en colecciones, y ausencia de estados de juego estructurados— aplicamos una reingeniería profunda del software. Esta transformación dio lugar a un sistema modular, robusto y testeable, que incorpora una campaña procedural de 30 niveles distribuida en tres biomas (Llanura, Hielo y Desierto de Atacama), una batalla climática final contra un Jefe OVNI en el espacio exterior, un modo competitivo Versus local en pantalla dividida y un desafío oculto de alta dificultad.

El propósito de este documento es proporcionar una radiografía exhaustiva de ingeniería de software que explique técnica y conceptualmente cómo y por qué fue diseñado cada componente del sistema:

- **Experiencia y Reglas de Dominio:** Análisis detallado de las mecánicas de juego, controles en múltiples plataformas, catálogo de elementos, penalizaciones físicas por deshidratación y modos especiales.
- **Evolución y Refactorización del Código:** Comparativa técnica de la transición desde una estructura monolítica hacia una arquitectura en capas desacopladas.
- **Modelado Estático (UML):** Diagramación formal de las clases del dominio, explicitando el uso de las abstracciones nativas de LibGDX y la separación entre lógica y renderizado.
- **Abstracciones Polimórficas:** Justificación e integración de la clase abstracta (`Gota`) y de la interfaz (`ControlMovimiento`), demostrando su rol activo en tiempo de ejecución.
- **Buenas Prácticas, GRASP y SOLID:** Análisis de encapsulamiento estricto, cohesión, bajo acoplamiento y cumplimiento de los cinco principios de diseño orientado a objetos.
- **Patrones de Diseño GoF:** Implementación justificada de cuatro patrones de diseño (**Singleton**, **Strategy**, **Template Method** y **Builder**), complementados con el contraste frente a sus respectivos antipatrones.
- **Control de Versiones y Trazabilidad:** Gestión del ciclo de desarrollo y repositorio del proyecto en Git y GitHub.

1. GM-1: ¿Qué experimenta el jugador?

El juego sitúa al usuario en el rol del **Guardián de la Lluvia**, un recolector que debe capturar agua pura y recursos vitales en un planeta afectado por la sequía, mientras esquiva contaminantes atmosféricos y se prepara para enfrentar a una nave extraterrestre invasora que drena los recursos hídricos.

El objetivo es claro y desafiante: atrapar las gotas y suministros beneficiosos, esquivar sustancias tóxicas o letales, recoger agua para evitar la fatiga a los 4 segundos y el daño desde los 12 segundos y sobrevivir a lo largo de una campaña de 30 niveles para librar una batalla final en el espacio.

¿Cómo se controla la aventura?

El jugador maneja al personaje principal (representado en el código por la clase `Personaje`) desplazándose horizontalmente por la parte inferior de la pantalla:

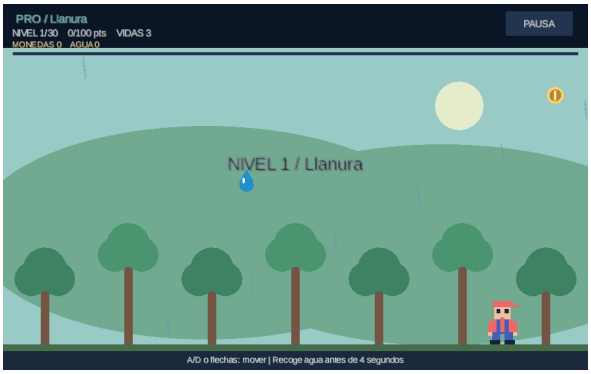
- **Modo Escritorio:** Movimiento con las teclas `A` y `D` o las flechas de dirección `LEFT` y `RIGHT`. En la batalla final contra el jefe, se dispara agua a presión presionando la `Barra Espaciadora` o haciendo clic izquierdo con el ratón.
- **Modo Táctil / Móvil:** El personaje sigue de forma continua la posición del dedo en pantalla mediante la proyección de coordenadas del `Viewport`. Al pulsar sobre el área de juego se ejecuta el disparo contra el jefe.
- **Pausa y Navegación:** La partida se pausa y reanuda con `P` o `ESC`. En Android, el botón físico `BACK` está capturado para volver al menú o pausar. La tecla `M` alterna el sonido y `ENTER` inicia la partida o confirma opciones.
- **Modo Versus Local (Teclado Compartido):** Dos jugadores compiten en pantalla dividida en el mismo monitor: el Jugador 1 se mueve con `A/D` y dispara con `W`; el Jugador 2 se mueve con flechas y dispara con flecha arriba (`UP`).

¿Qué tipos de gotas y objetos existen?

Cada elemento precipitado desde la atmósfera pertenece a un tipo bien diferenciado que altera la puntuación, las vidas o la movilidad:

Tabla 1.1: Tipos de elementos, sus efectos inmediatos y variación de estado

Elemento	Efecto al capturarlo	Variación
Agua Pura (GotaBuena)	Suma puntos, recarga tanque de agua y sacia la sed.	+10 pts, +1 agua
Agua Doble	Duplica la recarga de agua y puntaje.	+20 pts, +2 agua
Oasis	Hidratación profunda y gran bonificación.	+30 pts, +3 agua
Moneda Dorada	Moneda acumulable para comprar aspectos en la tienda.	+1 o +2 monedas
Café / Fruta	Incrementa la velocidad del personaje en 1,5× por 7 s.	Velocidad +50 %
Termo	Neutraliza y previene el congelamiento por 6 segundos.	Inmunidad térmica
Sangre	Contaminante biológico básico.	−20 puntos
Veneno	Intoxicación: reduce la velocidad del personaje a la mitad.	−10 pts, vel. −50 %
Nieve	Enfriamiento polar: ralentiza al personaje.	−20 pts, vel. −50 %
Hielo	Congelamiento severo: inmoviliza al jugador totalmente.	−30 pts, vel. = 0
Ácido	Químico corrosivo: resta vida e inflige daño.	−1 vida, −30 pts
Fuego / Cactus	Peligros ardientes y desérticos que causan daño directo.	−1 vida, daño



(a) Campaña en Bioma Llanura



(b) Campaña en Bioma Hielo



(c) Campaña en Desierto de Atacama



(d) Combate espacial contra el Jefe OVNI

Figura 1.1: Capturas del juego mostrando la evolución de biomas y la batalla climática final

¿Cómo cambia la dificultad y la progresión?

El juego no es estático; presenta una progresión estructurada y desafiante:

1. **Campaña de 30 Niveles:** Cada 100 puntos el jugador avanza de nivel. El juego transita por tres zonas climáticas: **Llanura** (niveles 1 a 10), **Hielo** (niveles 11 a 20) y **Desierto de Atacama** (niveles 21 a 30). Con cada nivel aumenta la velocidad de caída y la proporción de peligros.
2. **Mecánica de Sed y Penalización:** Pasados 4 segundos sin atrapar agua pura, la fatiga por deshidratación reduce la velocidad del personaje de forma progresiva, pudiendo llegar hasta un 60 % de penalización. Esto obliga a mantenerse en captura constante.
3. **Ronda de Monedas (Bonus Secreto):** Cada 5 niveles (5, 10, 15, 20, 25), la lluvia normal se interrumpe y se activa una ronda dorada de 8 segundos donde solo caen monedas para la tienda.
4. **Tres Niveles de Dificultad Inicial:**
   - *Noob:* 5 vidas, velocidad moderada y menor tasa de amenazas.

- *Pro:* 3 vidas, velocidad estándar y alta frecuencia de caída.
- *Hacker:* 1 sola vida (un impacto causa derrota inmediata) y máxima velocidad de reflejos.

¿Qué desafíos especiales y modos existen?

- **Ascenso y combate espacial.** Con 3.000 puntos comienza el Ascenso. El jefe entra en furia al 65 % de vida y despliega dos drones al 35 %. Mientras haya drones, su escudo bloquea los disparos y la nave se regenera. En los biomas y el Ascenso también se puede disparar contra las patrullas.
- **Desafío Oculto del Alien:** Un micro-botón secreto en el menú permite saltar directamente a combatir al OVNI en dificultad Pro con 40 unidades de agua. Este modo está aislado de las puntuaciones de campaña y su único propósito es desbloquear la skin secreta `Alien` si se logra la victoria.
- **Tienda de Aspectos (30 Skins):** Catálogo de personalización que incluye personajes como Mario, Sonic, Steve, Spiderman, Goku, Gojo y Alien, desbloqueables con monedas, alcanzando ciertos niveles en cada dificultad o venciendo al jefe.



(a) Tienda de personajes personalizables **(b)** Modo competitivo Versus local

Figura 1.2: Modalidades adicionales: Tienda de desbloqueos y Versus local en pantalla dividida

2. GM-2: ¿Cómo se transformó el código base?

¿De dónde partimos?

El proyecto se construyó sobre un tutorial básico de LibGDX que simulaba una lluvia de gotas atrapadas por un personaje. Ese prototipo, aunque funcional, presentaba una arquitectura espagueti: la lógica de juego, la gestión de recursos y el estado del jugador convivían en unas pocas clases sin separación de responsabilidades. Cualquier modificación requería reescribir múltiples partes y la adición de nuevos tipos de objetos resultaba casi imposible sin violar el principio de abierto/cerrado.

¿Qué decisiones de rediseño se tomaron?

Aplicamos una refactorización integral basada en patrones de diseño y principios SOLID. La siguiente tabla resume las diferencias fundamentales entre el prototipo base y la arquitectura desarrollada:

Tabla 2.1: Comparativa entre el prototipo base y la solución final implementada (GM)

Característica	Prototipo Base (LibGDX)	Sistema Implementado (GM)
Gestión del estado del juego	Variables sueltas en la clase principal o en el tarro; paso repetitivo de parámetros.	Clase `Partida` encapsula las reglas del juego. `ConfiguracionJuego` (Singleton) centraliza récord y audio.
Ciclo de vida de las gotas	Bloque condicional extenso en el bucle principal distinguiendo gotas manualmente.	Patrón **Template Method**: clase abstracta `Gota` define `actualizar()` como método plantilla `final` y `aplicarEfecto()` en subclases.
Eliminación en colecciones	Recorrido ascendente con eliminación que omitía elementos por desplazamiento de índice.	Recorrido inverso seguro (`for (int i = size - 1; i >= 0; i-)`) en `Lluvia.java`, garantizando estabilidad total.
Control de movimiento	Lectura fija de teclas acoplada directamente al modelo móvil.	Patrón **Strategy**: interfaz `ControlMovimiento` con implementaciones `ControlTeclado` y `ControlTactil`.
Configuración de dificultad	Parámetros fijos o repartidos en constantes numéricas aisladas.	Patrón **Builder**: `Nivel.Builder` construye etapas fluidamente. `Dificultad` actúa como Director de 30 niveles.
Estructura y renderizado	Código monolítico sin separación entre vista y lógica.	Separación estricta: `VistaJuego` gestiona el dibujo con `ShapeRenderer` y `SpriteBatch`. La lógica es testeable sin gráficos.

¿Cómo se organizan las responsabilidades ahora?

El sistema sigue una arquitectura modular en cuatro capas de responsabilidad:

1. **Capa de Modelo (Dominio):** Clases `Partida`, `Personaje`, `Lluvia`, `Gota`, `Nivel` y `JefeOvni`. No dependen de texturas, sonidos ni contextos gráficos de OpenGL; operan puramente sobre física, colisiones geométricas y reglas de negocio.

2. **Capa de Vista (Presentación):** Encapsulada en `VistaJuego`, auxiliada por `DibujoPersonaje`, `DibujoEscenario` y `DibujoObjetos`. Traduce el estado numérico del modelo en geometrías vectoriales limpias.
3. **Capa de Control (Orquestación y Entrada):** `GameLluvia` administra la máquina de 10 estados (`MENU`, `DIFICULTAD`, `TIENDA`, `PUNTUACIONES`, `AYUDA`, `JUGANDO`, `PAUSA`, `DERROTA`, `VICTORIA`, `RESULTADO_VERSUS`) y delega la captura de comandos en las estrategias de control.
4. **Capa de Persistencia:** `ConfiguracionJuego` y `PerfilJugador` interactúan con la interfaz `Preferences` de LibGDX para serializar monedas, puntuaciones y aspectos de manera segura.

¿Qué beneficios aporta esta nueva arquitectura?

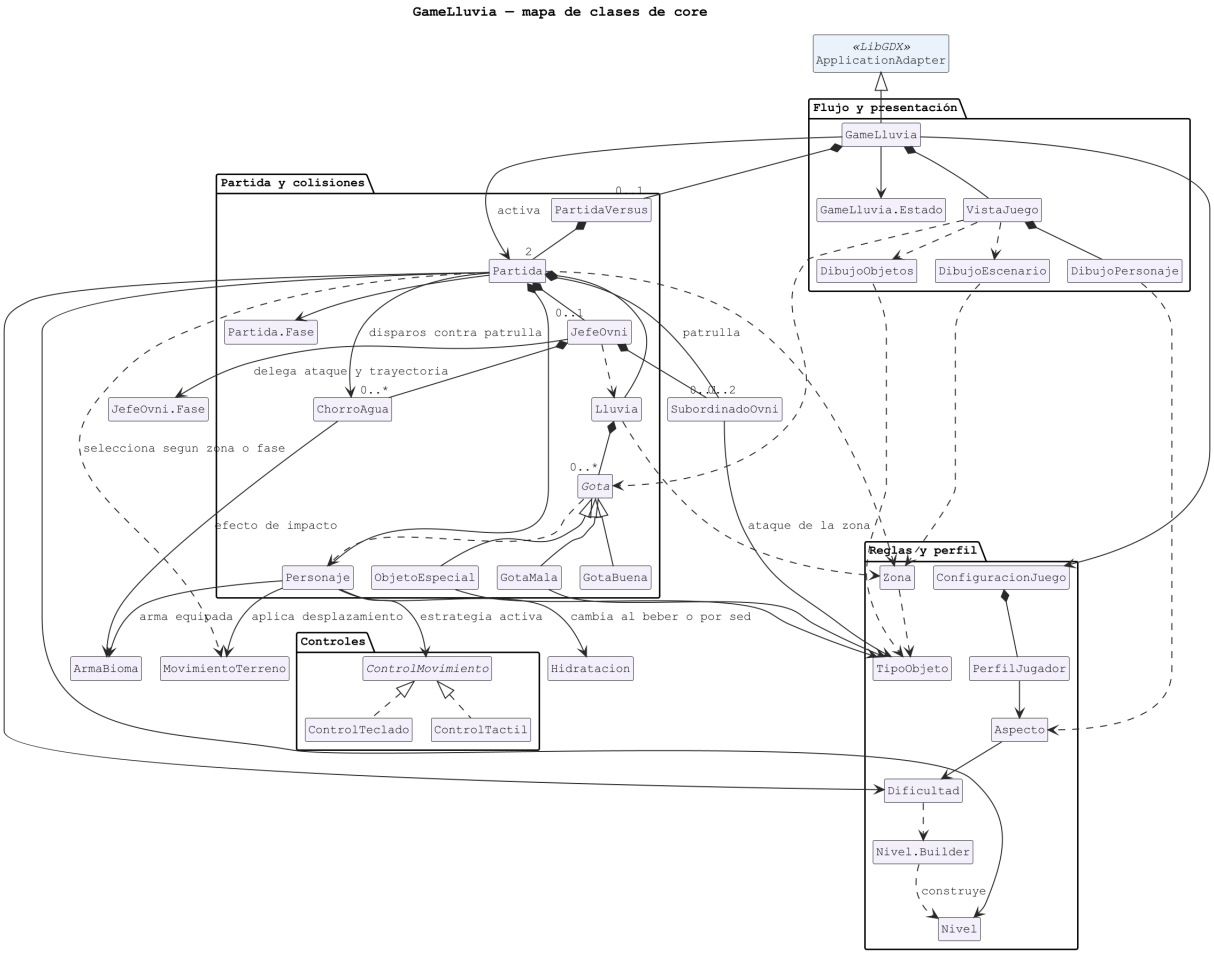
- **Extensibilidad:** Agregar una nueva gota o peligro ambiental solo requiere crear una subclase de `Gota` o un nuevo valor en `TipoObjeto` sin tocar el bucle de físicas de `Lluvia`.
- **Mantenibilidad:** Cada clase posee un propósito delimitado. Un ajuste visual en el aspecto del personaje se efectúa en `DibujoPersonaje` sin arriesgar la lógica de daño del `Personaje`.
- **Reutilización:** La misma clase `Partida` se reutiliza de forma transparente para la campaña individual, para cada una de las dos mitades del modo Versus local y para el desafío secreto del alien.
- **Facilidad de Pruebas Unitarias:** Al no requerir contexto OpenGL, las reglas del juego pudieron someterse a una batería de 36 pruebas unitarias automatizadas que se ejecutan en menos de un segundo en cualquier entorno de integración.

¿Dónde se evidencia esta transformación en el código?

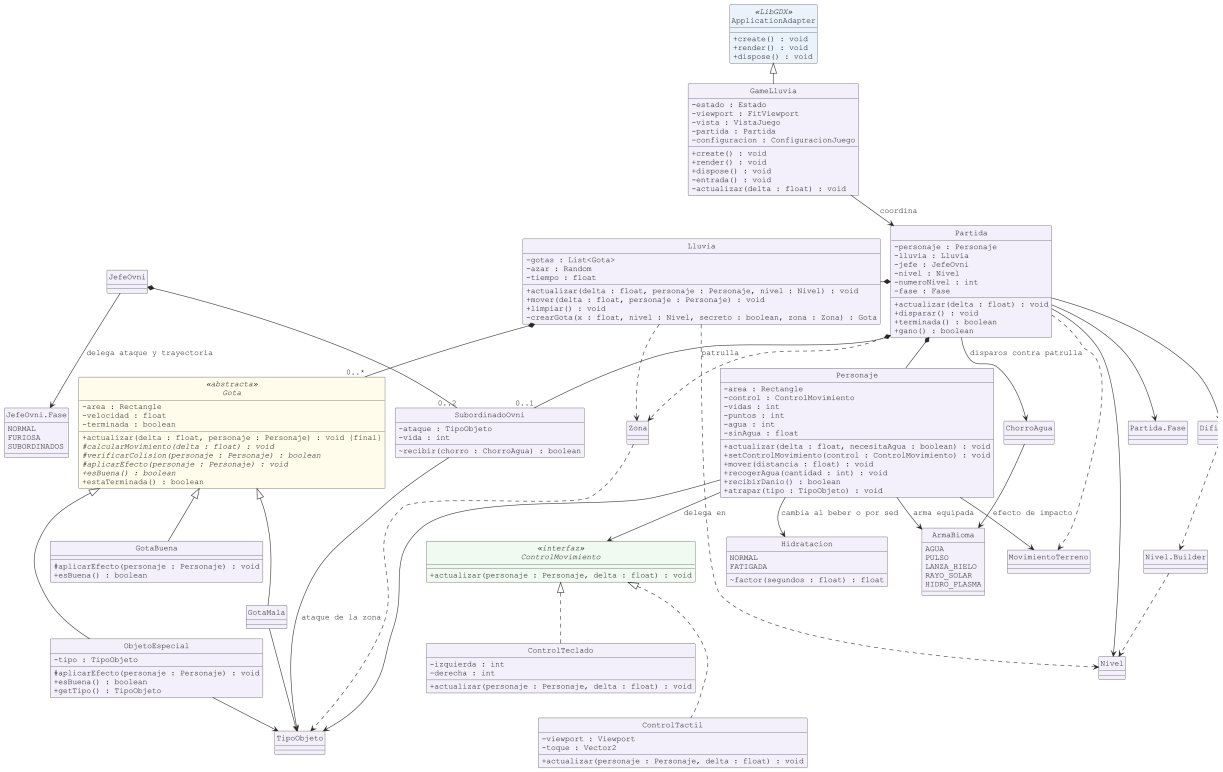
Los cambios están documentados en los archivos Java del proyecto. Los nombres de clases y métodos mencionados aparecen explícitamente en el código fuente, y sus relaciones se modelan en el diagrama UML de la sección GM-3. El lector puede verificar la implementación de cada patrón en los archivos `ConfiguracionJuego.java`, `Gota.java` y sus subclases, `ControlMovimiento.java` y sus implementaciones, y `Nivel.java` junto con su Builder.

3. Diagramas de clases (GM-3)

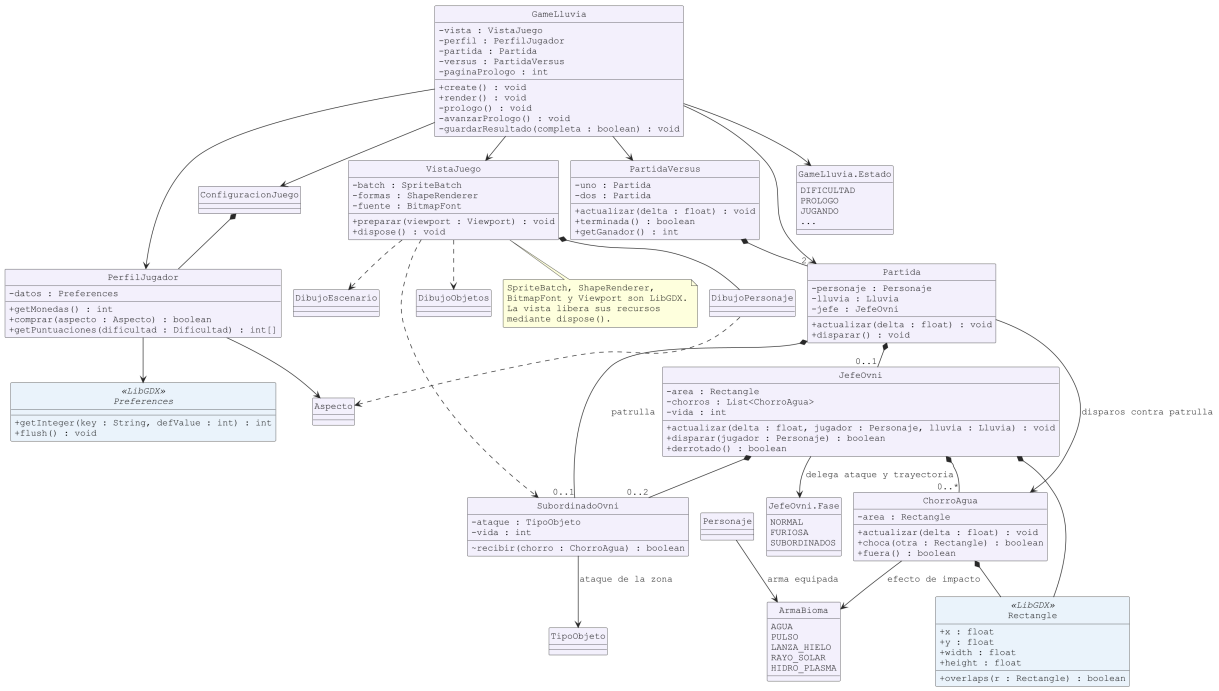
El maestro reúne los 29 tipos principales de `core`, incluidos los controles, la hidratación, las armas y los drones. Las dos vistas siguientes amplían el dominio y los sistemas.



3.1. Dominio: partida, lluvia y controles



3.2. Sistemas: presentación, combate y perfil



4. GM-4: ¿Qué abstracciones definen el comportamiento?

¿Por qué una clase abstracta para las gotas?

En el dominio del videojuego nunca cae una "gota genérica" sin propósito. Cada elemento que desciende de la atmósfera debe pertenecer a una variante concreta (agua pura, objeto especial con efectos, monedas o contaminantes) que determine su consecuencia al impactar al jugador.

Por esta razón se diseñó la clase `Gota` como abstracta. Ella encapsula los atributos y métodos comunes a todas las variantes, pero no puede ser instanciada directamente.

La clase abstracta define:

- **Atributos privados:** Delimitador de colisión (`Rectangle area`), tasa de caída vertical (`float velocidad`) y estado de ciclo de vida (`boolean terminada`).
- **Métodos concretos:** `estaTerminada()`, `getX()`, `getY()`.
- **El método plantilla `actualizar()`:** Declarado como `final`, fija la secuencia algorítmica de caída, escape inferior y colisión.
- **Operación primitiva abstracta:** `protected abstract void aplicarEfecto(Personaje p)`, delegada obligatoriamente en las clases hijas.

De esta forma, cualquier subclase hereda automáticamente la física de caída y colisión, debiendo únicamente ocuparse de programar su efecto particular.

¿Qué papel cumple la interfaz `ControlMovimiento`?

El desplazamiento del personaje depende del entorno en el que se ejecute el juego: en computadores se emplean teclas físicas, en dispositivos móviles se realiza arrastre táctil y en partidas multijugador dos usuarios comparten el mismo teclado con mapeos diferentes.

Para evitar que la clase `Personaje` se llene de condicionales dependientes del hardware, se definió la interfaz `ControlMovimiento`. Ella declara un único método: `actualizar(Personaje personaje, float delta)`.

¿Cuántas clases la implementan?

Actualmente existen dos implementaciones activas que operan en tiempo de ejecución:

1. **`ControlTeclado`:** Lee el estado de las teclas en `Gdx.input`. Posee sobrecarga de constructores: una variante por defecto que responde a `A`/`D` y flechas, y una variante parametrizada (`int izq, int der`) utilizada para instanciar a los dos contrincantes del modo Versus en un mismo teclado.

2. **`ControlTactil`:** Rastrea los toques en pantalla proyectando las coordenadas virtuales mediante la cámara (`viewport.unproject()`), desplazando suavemente al personaje hacia el dedo del usuario.

¿Cómo se evidencia esta jerarquía en el código?

Los archivos del proyecto evidencian claramente la utilización activa de estas estructuras:

- `Gota.java` declara: `public abstract class Gota`.
- `GotaBuena.java`, `GotaMala.java` y `ObjetoEspecial.java` extienden `Gota` e implementan `aplicarEfecto(Personaje)`, aportando cada una su propio comportamiento al colisionar con el jugador.
- `ControlMovimiento.java` define la interfaz. `ControlTeclado.java` y `ControlTactil.java` la implementan.
- En `GameLluvia.java`, se inyecta la estrategia de control correspondiente según la plataforma o el modo de juego al instanciar `Partida`.

¿Qué principios SOLID se respetan aquí?

- **Abierto/Cerrado (OCP):** La clase `Personaje` queda cerrada a modificaciones para añadir nuevos esquemas de control (por ejemplo, un mando USB); basta con crear una nueva clase que implemente `ControlMovimiento`.
- **Responsabilidad Única (SRP):** `Gota` gestiona cinemática y límites; cada subclase define su efecto particular; los controles solo interpretan hardware periférico.
- **Sustitución de Liskov (LSP):** Cualquier subclase de `Gota` puede sustituir a la clase base en la lista polimórfica de `Lluvia` sin quebrar la simulación.
- **Inversión de Dependencias (DIP):** `Personaje` depende de la abstracción `ControlMovimiento`, no de implementaciones concretas de teclado o librerías nativas.

¿Por qué este diseño es mejor que la alternativa con condicionales?

Si no se usara una clase abstracta ni interfaces, el código de `Lluvia` y `Personaje` tendría que preguntar el tipo de cada elemento con múltiples `instanceof` para decidir qué efecto aplicar. Eso generaría un alto acoplamiento y haría imposible añadir nuevas variedades sin modificar decenas de líneas. El diseño actual es limpio, extensible y fácil de probar.

5. GM-5: ¿Cómo se protegen los datos y se organiza el código?

¿Qué significa encapsulamiento en este proyecto?

El encapsulamiento es uno de los pilares de la orientación a objetos. En el juego, todas las clases del modelo declaran sus atributos como `private` o `protected`, ofreciendo métodos públicos únicamente cuando es estrictamente necesario bajo rigurosas reglas de negocio. Esto evita que clases externas modifiquen el estado interno de forma arbitraria o inconsistente.

Por ejemplo, la clase `Personaje` posee atributos vitales como `vidas`, `puntos`, `agua`, `sinAgua` e `invulnerabilidad` que son completamente privados. Su modificación solo es posible a través de métodos de negocio controlados:

- `recibirDanio()`: Comprueba que el personaje no esté en periodo de invulnerabilidad (`invulnerabilidad == 0`). Si es impactado, reduce exactamente una vida y activa una protección temporal de 1.2 segundos (`invulnerabilidad = 1.2f`).
- `quitarPuntos(int cantidad)`: Asegura mediante `Math.max(0, puntos - cantidad)` que el puntaje jamás adopte valores negativos.
- `colocar(float x)`: Emplea `MathUtils.clamp(x, 0, anchoMundo - ANCHO)`, garantizando matemáticamente que el personaje nunca pueda desplazarse fuera del área visible de la pantalla.
- **No exposición de referencias mutables:** `Personaje` no entrega su objeto geométrico interno `Rectangle area`. Para las colisiones expone el método booleano `choca(Rectangle otra)`, resguardando su integridad física. Asimismo, en `Lluvia.java`, la colección de gotas se expone a la vista mediante `Collections.unmodifiableList(gotas)`, imposibilitando que la vista altere la lista de entidades activas.

¿Qué principios SOLID y patrones GRASP se aplicaron?

El diseño arquitectónico adoptó los cinco principios SOLID y las directrices GRASP formuladas por Craig Larman:

1. **Experto en Información (GRASP):** Cada clase custodia sus propios datos. `Personaje` es el experto que calcula su propia velocidad (`getVelocidad()`), combinando la tasa base con penalizaciones por sed y estados alterados (hielo, café, veneno). `JefeOvni` conoce su umbral de furia al 50 % de salud. `PerfilJugador` administra las tablas de récords.
2. **Alta Cohesión y Bajo Acoplamiento (GRASP):** `Lluvia` se ocupa exclusivamente de la vida y muerte de las gotas. `VistaJuego` se dedica solo al dibujado geométrico

en pantalla. `Partida` gestiona los niveles y el avance sin conocer librerías de sonido ni de texturas.

3. **Responsabilidad Única (SRP):** Cada clase posee una única razón de cambio bien acotada. La lógica de daño no depende de OpenGL ni del subsistema de audio.
4. **Abierto/Cerrado (OCP):** Incorporar un nuevo control o un nuevo tipo de objeto especial no requiere reescribir la lógica de `Personaje` ni la máquina de físicas de `Lluvia`.
5. **Sustitución de Liskov (LSP):** Todas las subclases de `Gota` (`GotaBuena`, `GotaMala`, `ObjetoEspecial`) son sustituibles en el método de actualización sin alterar la consistencia del bucle de juego.
6. **Segregación de Interfaces (ISP):** La interfaz `ControlMovimiento` es limpia y mínima: exige un único método (`actualizar()`), evitando imponer métodos innecesarios a los clientes.
7. **Inversión de Dependencias (DIP):** El modelo de dominio de alto nivel (`Personaje`) depende de la abstracción de control (`ControlMovimiento`) y no de implementaciones concretas de hardware.

¿Dónde se evidencian estas buenas prácticas en el código?

- En `Personaje.java`, todos los atributos son privados y su modificación se realiza únicamente mediante métodos de negocio como `recibirDanio()` o `quitarPuntos()`.
- En `ConfiguracionJuego.java`, el constructor es privado y el acceso se restringe a través del método estático `getInstancia()`.
- En `Lluvia.java`, se retorna una vista inmutable de la colección de entidades para proteger la integridad de las gotas activas.

¿Qué beneficios concretos aporta esta organización?

- **Extensible:** Añadir nuevos personajes a la tienda simplemente requiere agregar una entrada al enumerador `Aspecto.java` sin modificar la estructura del jugador.
- **Mantenible:** Un fallo o ajuste en la velocidad por deshidratación se corrige puntualmente en `Personaje.getVelocidad()` sin riesgo de generar efectos colaterales en la detección de colisiones.
- **Verificable mediante pruebas unitarias:** La lógica no posee acoplamientos ocultos ni variables globales no controladas, lo que permitió validar el 100 % de las 36 pruebas automatizadas del proyecto sin errores.

6. GM-6: ¿Por qué centralizar la configuración y el audio? (Singleton)

¿Qué problemas presentaba el código base original?

En el prototipo inicial de LibGDX, las variables de configuración (como el estado del audio y las mejores puntuaciones) debían ser pasadas manualmente como referencias a través de múltiples clases o se leían de forma descontrolada desde el disco.

Cada vez que una pantalla o estado del juego necesitaba silenciar la música o consultar el récord, requería recibir una copia del objeto o invocar llamadas de I/O dispersas. Esto generaba un alto acoplamiento, duplicación de instancias de preferencias y el riesgo constante de desincronizar el audio: una pantalla podía silenciar los efectos mientras otra mantenía una referencia antigua que reanudaba la música de fondo.

¿Cómo resuelve el patrón Singleton estos problemas?

El patrón Singleton garantiza que una clase tenga una única instancia en toda la máquina virtual y proporciona un punto de acceso global y controlado hacia ella. En el proyecto se implementó en la clase `ConfiguracionJuego`:

1. **Constructor Privado:** Impide la creación de instancias mediante el operador `new` desde cualquier otra clase del proyecto.
2. **Instancia Estática Única:** Se mantiene una variable estática privada (`private static ConfiguracionJuego instancia;`) que almacena la única referencia existente.
3. **Punto de Acceso Global:** El método público estático `getInstancia()` inicializa la clase la primera vez que es solicitado (*Lazy Initialization*) y entrega dicha referencia compartida a cualquier cliente.
4. **Encapsulación de Almacenamiento Local:** Administra de forma atómica el archivo de preferencias mediante `Preferences` de LibGDX y delega la gestión de aspectos y monedas en `PerfilJugador`.

¿Dónde se evidencia su implementación en el código?

- En `ConfiguracionJuego.java`, el constructor es privado y la instancia única se obtiene mediante el método estático `getInstancia()`. Centraliza el estado del sonido, el récord y el perfil del jugador.
- En `GameLluvia.java`, durante la inicialización en `create()`, se accede a la configuración invocando `ConfiguracionJuego.getInstancia()`, asegurando una fuente única de verdad para el audio y las preferencias.
- Al presionar la tecla de audio o registrar un nuevo récord, cualquier parte del juego interactúa directamente con los métodos de negocio de la instancia única, reflejando

el cambio de inmediato en todo el sistema.

¿Qué principios SOLID se refuerzan con este patrón?

- **Responsabilidad Única (SRP):** `ConfiguracionJuego` se ocupa exclusivamente de centralizar las preferencias globales del usuario y sincronizar el estado del audio, liberando a las clases de vista y de juego de responsabilidades de persistencia de bajo nivel.
- **Acceso global controlado:** Se previene el paso redundante de referencias por constructores a lo largo de diez pantallas y subclases.

¿Qué hubiera pasado si no se usara Singleton?

Sin el patrón Singleton, cada pantalla (`MENU`, `DIFICULTAD`, `TIENDA`, `JUGANDO`, `PAUSA`) tendría que recibir el objeto de preferencias por parámetro o instanciar su propio lector de `Preferences`. Esto ocasionaría:

1. Lecturas redundantes de disco ralentizando los cambios de escena.
2. Condiciones de carrera al sobreescribir el archivo de guardado local entre partidas.
3. Un diseño altamente acoplado donde modificar el almacenamiento obligaría a cambiar la firma de constructores en todas las pantallas del juego.

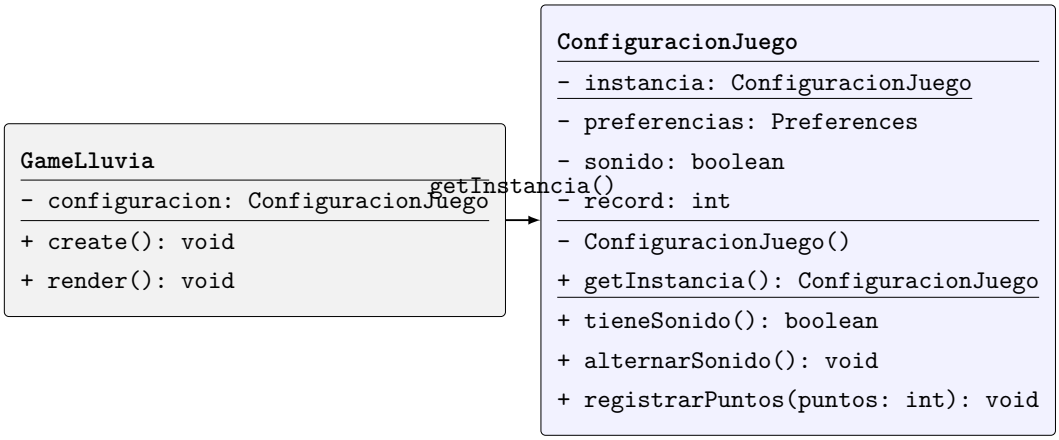


Figura 6.1: Diagrama UML del patrón Singleton aplicado a la configuración global y audio

7. GM-7: ¿Cómo se desacoplan los controles de entrada? (Strategy)

¿Qué limitación tenían los controles en el código base?

En el prototipo original, el tarro leía directamente las teclas de flecha (`Gdx.input.isKeyPressed(Input.Keys.LEFT)`) dentro de su propio método de actualización. Esta ligadura rígida impedía adaptar el juego a dispositivos móviles con pantalla táctil, imposibilitaba crear esquemas alternativos (como las teclas `A` y `D`) e impedía rotundamente implementar un modo multijugador local donde dos jugadores debieran compartir el mismo teclado físico con asignaciones de teclas independientes.

Si se hubiera querido añadir soporte táctil o Versus sin patrones, habría sido necesario llenar la clase del personaje con múltiples banderas booleanas y bloques condicionales dependientes de la plataforma.

¿Cómo resuelve el patrón Strategy esta situación?

El patrón Strategy define una familia de algoritmos, los encapsula en clases separadas y los hace intercambiables en tiempo de ejecución. En el juego se implementó para desacoplar el procesamiento cinemático del jugador de los dispositivos físicos de entrada:

1. **Estrategia Abstracta (`ControlMovimiento`):** Interfaz que declara el método común `actualizar(Personaje personaje, float delta)`.
2. **Estrategias Concretas:**
   - `ControlTeclado`: Implementa la lectura de teclas. Gracias a su sobrecarga de constructores, permite operar en modo estándar (`A`/`D` y flechas) o en modo multijugador parametrizando las teclas de cada jugador.
   - `ControlTactil`: Implementa la lectura de toques en pantalla proyectando el punto del puntero mediante la cámara virtual del juego (`viewport.unproject()`).
3. **Contexto (`Personaje`):** Mantiene una referencia a la interfaz y delega en ella el cálculo del desplazamiento en cada cuadro (`control.actualizar(this, delta)`), sin saber si el comando proviene de un teclado, una pantalla táctil o un script simulado.

¿Dónde se observa esta implementación en el código?

- `ControlMovimiento.java`: define la interfaz con el método `actualizar()`.
- `ControlTeclado.java`: implementa el desplazamiento por teclado, soportando teclas individuales para el modo individual o para cada jugador en el modo Versus.
- `ControlTactil.java`: implementa el cálculo cinemático hacia el punto tocado proyectando las coordenadas en el viewport.

- En `GameLluvia.alternarControlEnPartida()`, T o el botón del HUD llaman a `Personaje.setControlMovimiento()`. En Versus cada jugador mantiene su teclado. `Partida` actualiza el movimiento por terreno, `Personaje` cambia entre movimiento normal y fatiga, y el jefe cambia de estrategia en cada fase.

¿Qué principios SOLID se benefician con este patrón?

- **Abierto/Cerrado (OCP):** La clase `Personaje` queda cerrada a modificaciones para dar soporte a nuevos dispositivos. Para incorporar un mando o joystick, basta con crear una nueva clase que implemente `ControlMovimiento`.
- **Responsabilidad Única (SRP):** La cinemática del personaje se aísla de la lectura de eventos de hardware de LibGDX.
- **Inversión de Dependencias (DIP):** El personaje depende exclusivamente de una abstracción (`ControlMovimiento`), no de clases concretas de bajo nivel.

¿Cómo se vería el código sin Strategy?

Sin este patrón, la clase `Personaje` tendría un enumerador o variables booleanas para decidir el movimiento con una estructura `if-else` o `switch` dentro de su método de actualización. Cada nuevo esquema de entrada obligaría a modificar directamente al personaje, aumentando la probabilidad de errores y violando el principio de Abierto/Cerrado. Con Strategy, cada algoritmo vive en su propia clase y puede verificarse de forma aislada.

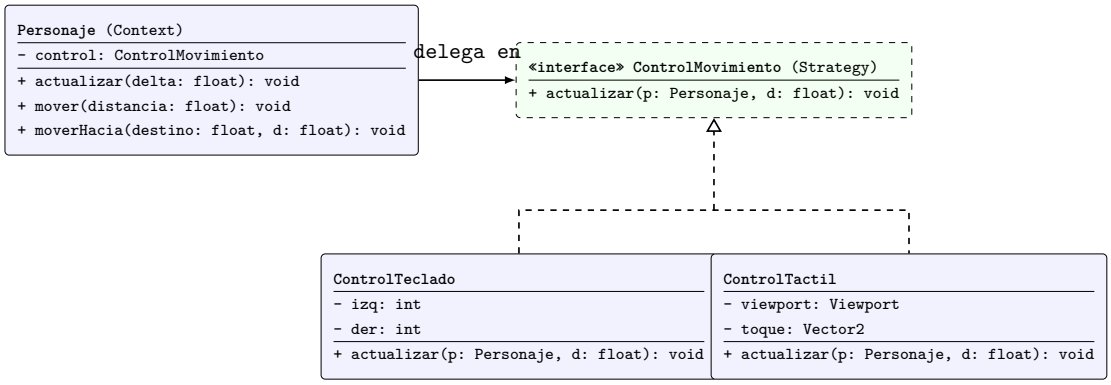


Figura 7.1: Diagrama UML del patrón Strategy aplicado al control de movimiento

8. GM-8: ¿Cómo se define el esqueleto de la caída y captura? (Template Method)

¿Qué problema presentaba la lógica de colisión en el código base?

En el prototipo original, el movimiento y la colisión de las gotas se gestionaban dentro de un extenso bloque condicional en el bucle principal de `Lluvia`. Cada tipo de gota requería una rama `if-else` que consultaba si la gota impactaba el tarro, aplicaba manualmente el efecto (sumar puntos o restar vidas) y luego la eliminaba de la colección.

Esta aproximación provocaba una severa duplicación de código geométrico, acoplaba fuertemente la clase gestora `Lluvia` a los tipos específicos de gotas y violaba el principio de Abierto/Cerrado: añadir un nuevo elemento (como agua doble, hielo o fuego) obligaba a modificar y recompilar el bucle central del juego.

¿Cómo resuelve el patrón Template Method esta situación?

El patrón Template Method define los pasos invariantes de un algoritmo en una operación de la clase base, postergando la definición de ciertos pasos variables a las subclases mediante métodos abstractos o "gancho". En el juego, la clase abstracta `Gota` implementa el método `actualizar()` con el modificador `final`, estableciendo una secuencia invariable que se ejecuta en cada cuadro de simulación:

1. **Paso 1 (Caída lineal):** Decrementa la coordenada vertical según la velocidad de la gota y el delta de tiempo: `area.y -= velocidad * delta`.
2. **Paso 2 (Escape de pantalla):** Verifica si la gota rebasó el borde inferior; en tal caso, marca `terminada = true` para que sea liberada.
3. **Paso 3 (colisión y efecto).** Llama a `verificarColision()`. Si hay contacto, aplica el efecto, ejecuta `despuesDeAtrapar()` y marca la gota como terminada.

De esta forma, la secuencia física permanece invariable y centralizada en la clase madre, mientras que la consecuencia específica (curar, dañar, hidratar o alterar la velocidad) queda encapsulada en cada subclase concreta.

¿Dónde se evidencia esta implementación en el código?

- En `Gota.java`, el método `actualizar()` está declarado como `public final void actualizar(float delta, Personaje personaje)`. Dentro de él se ejecuta el movimiento, la salida de pantalla y la llamada a `aplicarEfecto(personaje)`.
- `GotaBuena`: el agua doble cae más despacio, usa un margen de contacto de dos unidades y da dos cargas. El aviso se muestra desde el hook.
- `GotaMala.java`: implementa `aplicarEfecto()` invocando `personaje.recibirDaño()`.

- `ObjetoEspecial.java`: implementa `aplicarEfecto()` delegando en `personaje.atrapar(tipo)`.
- En `Lluvia.java`, dentro del bucle de colisiones, se invoca polimórficamente `gota.actualizar(delta, personaje)` sin distinguir el tipo concreto de elemento.

¿Qué principios SOLID se favorecen con este patrón?

- **Responsabilidad Única (SRP).** La clase base fija el orden y las subclases personalizan movimiento, colisión y efecto.
- **Abierto/Cerrado (OCP).** Una variante puede cambiar los pasos sin reescribir el ciclo general. Para que aparezca en una zona se configura su generación.
- **Sustitución de Liskov (LSP):** Todas las subclases pueden ser tratadas como `Gota` y el método plantilla funciona correctamente sin importar la subclase concreta.

¿Qué pasaría si no se usara Template Method?

Sin este patrón, la clase `Lluvia` tendría que contener una estructura condicional que pregunte el tipo de cada gota (`instanceof` o un enumerador) para aplicar el efecto correspondiente. Cada nuevo tipo obligaría a añadir otra rama `if-else`, haciendo el código cada vez más extenso, propenso a errores y difícil de mantener. El Template Method encapsula la variación donde corresponde: dentro de cada subclase.

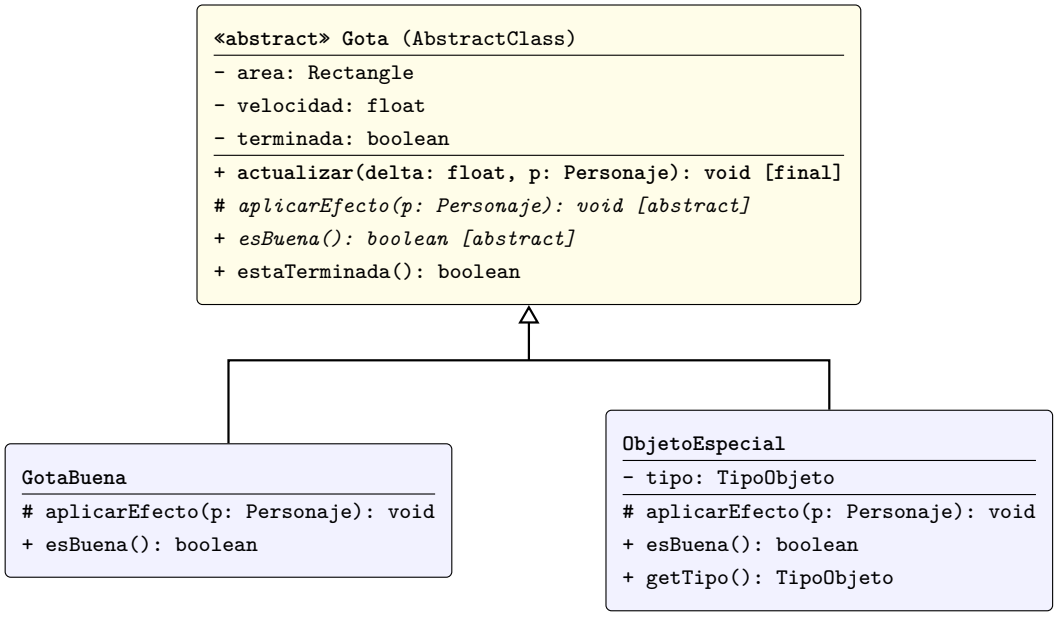


Figura 8.1: Diagrama UML del patrón Template Method en la jerarquía Gota

9. GM-9: ¿Cómo se construyen niveles complejos de forma legible? (Builder)

¿Qué dificultad presentaba la creación de niveles en el código base?

En el prototipo original, los parámetros de dificultad (como velocidades de caída, intervalos de tiempo y probabilidad de gotas malas) eran constantes numéricas fijas o estaban dispersas en varias líneas dentro de la clase `Lluvia`. Para escalar el juego y soportar una campaña de 30 niveles procedurales dividida en 3 dificultades (`NOOB`, `PRO`, `HACKER`), se requería configurar múltiples valores por cada nivel: nombre, umbral acumulado de puntos, velocidad de caída, intervalo entre gotas y probabilidad estadística de amenazas.

Una aproximación ingenua habría sido crear un constructor con cinco o más parámetros en la clase `Nivel`:

```
1 // Antipatron del Constructor Telescopico
2 new Nivel("HACKER / 25", 2500, 580f, 0.22f, 0.42f);
```

Este enfoque produce el conocido antipatrón del Constructor Telescópico. Resulta sumamente propenso a errores humanos involuntarios (como intercambiar por accidente la velocidad con el intervalo o la probabilidad), dificulta la lectura del código y no provee validación antes de crear el objeto.

¿Cómo resuelve el patrón Builder este problema?

El patrón Builder separa la construcción de un objeto complejo de su representación final, permitiendo crear diferentes configuraciones paso a paso mediante una interfaz fluida (*fluent interface*).

En el proyecto se diseñó la clase estática interna `Nivel.Builder` (`Nivel.java`, líneas 23-43):

1. Se crea una instancia del Builder especificando el nombre identificador del nivel.
2. Se invocan los métodos de configuración deseados (`objetivo()`, `velocidad()`, `intervalo()`, `probabilidadMala()`), cada uno de los cuales retorna `this` para permitir el encadenamiento fluido.
3. Finalmente, se invoca el método terminal `construir()`, el cual valida exhaustivamente las invariantes numéricas (valores positivos finitos y probabilidades entre 0 y 1). Si los datos son válidos, instancia el objeto inmutable `Nivel`.
4. **Rol de Director en `Dificultad`:** El enum `Dificultad` actúa como *Director* formal, encapsulando las fórmulas matemáticas de progresión y configurando el Builder para generar automáticamente los 30 niveles.

¿Dónde se evidencia esta implementación en el código?

- `Nivel.Builder`: clase estática anidada que mantiene los parámetros de la etapa y ofrece métodos encadenables (`objetivo()`, `velocidad()`, `intervalo()`, `probabilidadMala()`) que retornan `this`.
- `Nivel.java`: su constructor es privado y recibe una instancia de `Builder`, garantizando que ninguna clase externa pueda instanciar un nivel sin pasar por el proceso de construcción y validación.
- En `Dificultad.java`, el método `crearNivel(numero)` utiliza el builder de forma declarativa:

```
new Nivel.Builder(name() + " / " + numero)
    .objetivo(numero * 100)
    .velocidad(Math.min(620, velocidad + (numero - 1) * 15))
    .intervalo(Math.max(0.2f, intervalo - (numero - 1) * 0.015f))
    .probabilidadMala(Math.min(0.48f, peligro + (numero - 1) * 0.01f))
    .construir();
```

¿Qué principios SOLID se refuerzan con este patrón?

- **Responsabilidad Única (SRP):** La clase `Nivel` solo almacena los parámetros inmutables de una etapa. La lógica de ensamblado y validación de consistencia reside exclusivamente en `Nivel.Builder`.
- **Segregación de Interfaces implícita:** El cliente solo interactúa con los métodos encadenados que necesita modificar; los parámetros no especificados conservan valores por defecto estables.
- **Legibilidad y Mantenibilidad:** La creación de niveles mediante nombres de métodos descriptivos se documenta por sí misma y previene la inversión involuntaria de argumentos.

¿Cómo se vería el código sin Builder?

Sin este patrón, se necesitaría un constructor telescópico como `new Nivel("HACKER / 25", 2500, 580f, 0.22f, 0.42f)`. Al invocarlo, el desarrollador tendría que recordar el orden exacto de múltiples parámetros numéricos contiguos del mismo tipo, lo que propicia confusiones y errores silenciosos en tiempo de ejecución. Además, para crear los 30 niveles se repetiría la misma estructura rígida sin validación previa. El Builder elimina estos problemas encapsulando la construcción paso a paso.

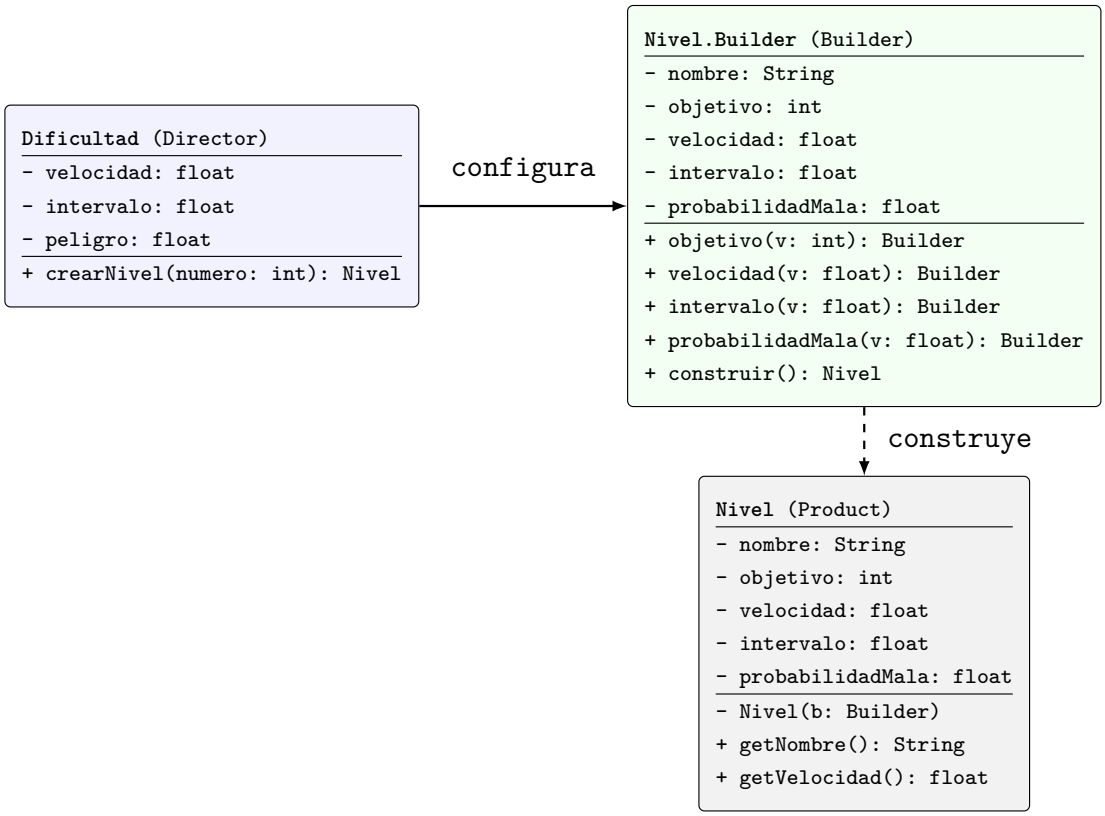


Figura 9.1: Diagrama UML del patrón Builder con Director en Dificultad

10. Control de Versiones y Trazabilidad en GitHub (GM-10)

El ciclo de desarrollo del proyecto se gestionó mediante el sistema de control de versiones distribuido Git y la plataforma remota GitHub, manteniéndose disponible en el repositorio público oficial:

`https://github.com/maverick1006/Proyecto-Game-LLuvia-`

11. Recorrección de Guardián de la Lluvia

Luma explica que una nave contaminó la lluvia. El recolector del juego original representa la reserva del Guardián: las gotas azules lo hidratan y se convierten en munición. La llanura ofrece el Pulso, el hielo la Espada de Hielo, Atacama la Pistola Láser Solar y el Ascenso puede dejar caer el Cañón Hidro-Plasma. El encuentro oculto conserva el aspecto Alien como recompensa.

Tras cuatro segundos sin recoger agua, el Guardián pierde velocidad de forma progresiva. A los doce segundos recibe daño y luego cada cuatro segundos. Una gota normal o doble reinicia la sed. La doble, marcada x2, da dos unidades de agua y dos puntos de carga. La barra se llena con seis puntos desde que se equipa el arma. El siguiente disparo activa el poder especial y vacía la barra. La espada prolonga el frío, el láser deja más pulsos de calor y el cañón lanza tres chorros.

El jefe tiene 72, 90 o 108 puntos de vida según la dificultad. Empieza con ataques regulares. Al 65 % cambia su trayectoria y su cadencia, con una segunda gota cada tercer ataque. Al 35 % despliega dos drones. Mientras quede uno, el escudo bloquea los impactos y la nave recupera una vida por segundo hasta la mitad de su máximo. Cuando cae el último, vuelve a ser vulnerable.

11.1. Strategy y Template Method

En el juego base, los arreglos separados exigían mantener sincronizados los índices de las gotas. En esta versión, `Lluvia` recorre una colección de objetos y cada subtipo resuelve su comportamiento. La partida decide los cambios de zona y el personaje conserva su estado cuando cambia de control.

En Strategy, T o el botón del HUD llaman a `setControlMovimiento()`. `Personaje.actualizar()` selecciona la fatiga al superar cuatro segundos, y `recogerAgua()` restablece la estrategia normal. El arma se equipa al recogerla y `JefeOvni.disparar()` usa su comportamiento normal o cargado. `JefeOvni.recibirDaño()` cambia la fase en vivo y `actualizar()` delega la trayectoria y el ataque a la fase activa. Esto permite cambiar cada comportamiento en la misma partida y mantener sus datos.

En Template Method, `Gota.actualizar()` fija el orden de movimiento, comprobación de colisión y efecto. Después llama a `despuesDeAtrapar()`, que el agua doble usa para mostrar su aviso. Las gotas simples, dobles, malas y especiales pueden ajustar la caída, el área de contacto y el efecto. La clase base mantiene el descarte y evita que un mismo objeto aplique su efecto dos veces.

11.2. Singleton y Builder

`ConfiguracionJuego.getInstancia()` comparte el ajuste de sonido y el perfil entre pantallas. Así, una decisión del menú se refleja en la partida. `Dificultad.crearNivel()`

usa `Nivel.Builder` para indicar el objetivo, la velocidad, el intervalo y la probabilidad de peligro antes de construir el nivel. `Partida` usa esa configuración al avanzar.

11.3. Luma y las patrullas de la nave

Luma guía al jugador en el prólogo y lleva un traje turquesa con capucha y un escudo marcado con una gota. En la tienda cuesta 240 monedas. El traje es cosmético y el escudo que absorbe golpes se recoge durante la partida.

Las patrullas lanzan ácido en la llanura, nieve en Hielo y fuego en Atacama. El Ascenso tiene una patrulla de ácido. `Partida.actualizarPatrulla()` actualiza la nave y resuelve los impactos. `Partida.disparar()` usa el arma equipada y activa su especial si la barra está llena. Durante el jefe delega el disparo en `JefeOvni`. Las patrullas son opcionales: avanzar depende del puntaje, y el tramo secreto conserva solo monedas.